

Introduction to Neural Networks

Learning from Scratch

by Rishikesh Yadav

June 03, 2026

Assistant Professor, School of Mathematical and Statistical Sciences, IIT Mandi, India

Table of Contents

- 1. Recap: Regression and Classification
- 2. Neural Networks
 - 2.1 Intro to Neural Networks
 - 2.2 Components of Neural Networks
 - Layers
 - Neurons, Weights, and Biases
 - Activation Functions
 - 2.3 Forward Propagation: Making Predictions
 - 2.4 Training of NNs (Learning the Weights): Backpropagation
 - Recall
 - Example: Backpropagation for two layers NNs
 - 2.5 Building Your Very First Neural Network From Scratch
 - 2.6 Fitting the Neural Network using Tensorflow

Course Objectives

The objectives of this lecture would be:

- Recap **regression** and **classification** techniques.
- Understand key ideas from scratch in **neural networks**.
- Build and train simple **neural networks** from scratch and also using **TensorFlow**.
- Bridge **traditional statistical regression methods** with **neural network**.

Some Good References

- Bishop, C. M. (2007). *Pattern Recognition and Machine Learning*. Springer.
- Géron, A. (2022). *Hands-On Machine Learning with Scikit-Learn, Keras, and TensorFlow* (3rd ed.). O'Reilly. [[GitHub companion code](#)]
- Goodfellow, I., Bengio, Y., & Courville, A. (2016). *Deep Learning*. MIT Press. [[Free online](#)]
- Chollet, F. (2021). *Deep Learning with Python* (2nd ed.). Manning.
- Nielsen, M. A. (2015). *Neural Networks and Deep Learning*. [[Free online book](#)]
- Online learning platforms:
 - [Coursera - Deep Learning Specialization](#)
 - [fast.ai - Practical Deep Learning for Coders](#)
 - [DeepLearning.AI](#)

Recap: Regression and Classification

Supervised Learning: General Framework

- **Goal:** Learn (estimate) a function $f(\mathbf{x}; \mathbf{w})$ that maps several input features $\mathbf{x}$ to an output variable y .

$$y \approx f(\mathbf{x}; \mathbf{w}) \quad (\text{there will be some error terms as well})$$

Supervised Learning: General Framework

- **Goal:** Learn (estimate) a function $f(\mathbf{x}; \mathbf{w})$ that maps several input features $\mathbf{x}$ to an output variable y .

$$y \approx f(\mathbf{x}; \mathbf{w}) \quad (\text{there will be some error terms as well})$$

- **Data:** A collection of input–output pairs

$$\mathcal{D} = \{(\mathbf{x}_i, y_i)\}_{i=1}^n$$

Supervised Learning: General Framework

- **Goal:** Learn (estimate) a function $f(\mathbf{x}; \mathbf{w})$ that maps several input features $\mathbf{x}$ to an output variable y .

$$y \approx f(\mathbf{x}; \mathbf{w}) \quad (\text{there will be some error terms as well})$$

- **Data:** A collection of input–output pairs

$$\mathcal{D} = \{(\mathbf{x}_i, y_i)\}_{i=1}^n$$

- Inputs (features): $\mathbf{x}_i = (x_{i1}, x_{i2}, \dots, x_{ip})^\top$

Supervised Learning: General Framework

- **Goal:** Learn (estimate) a function $f(\mathbf{x}; \mathbf{w})$ that maps several input features $\mathbf{x}$ to an output variable y .

$$y \approx f(\mathbf{x}; \mathbf{w}) \quad (\text{there will be some error terms as well})$$

- **Data:** A collection of input–output pairs

$$\mathcal{D} = \{(\mathbf{x}_i, y_i)\}_{i=1}^n$$

- Inputs (features): $\mathbf{x}_i = (x_{i1}, x_{i2}, \dots, x_{ip})^\top$
- Outputs (targets): y_i (numeric or categorical)

Supervised Learning: General Framework

- **Goal:** Learn (estimate) a function $f(\mathbf{x}; \mathbf{w})$ that maps several input features $\mathbf{x}$ to an output variable y .

$$y \approx f(\mathbf{x}; \mathbf{w}) \quad (\text{there will be some error terms as well})$$

- **Data:** A collection of input–output pairs

$$\mathcal{D} = \{(\mathbf{x}_i, y_i)\}_{i=1}^n$$

- Inputs (features): $\mathbf{x}_i = (x_{i1}, x_{i2}, \dots, x_{ip})^\top$
- Outputs (targets): y_i (numeric or categorical)
- **Learning:** Estimate the unknown parameters (weights in machine learning) $\mathbf{w}$ by minimizing a suitable loss function $\mathcal{L}(\mathbf{w})$.

Supervised Learning: General Framework

- **Goal:** Learn (estimate) a function $f(\mathbf{x}; \mathbf{w})$ that maps several input features $\mathbf{x}$ to an output variable y .

$$y \approx f(\mathbf{x}; \mathbf{w}) \quad (\text{there will be some error terms as well})$$

- **Data:** A collection of input–output pairs

$$\mathcal{D} = \{(\mathbf{x}_i, y_i)\}_{i=1}^n$$

- Inputs (features): $\mathbf{x}_i = (x_{i1}, x_{i2}, \dots, x_{ip})^\top$
- Outputs (targets): y_i (numeric or categorical)
- **Learning:** Estimate the unknown parameters (weights in machine learning) $\mathbf{w}$ by minimizing a suitable loss function $\mathcal{L}(\mathbf{w})$.
- **Prediction:** Once $\hat{\mathbf{w}}$ is obtained, use

$$\hat{y} = f(\mathbf{x}_0; \hat{\mathbf{w}})$$

for new input $\mathbf{x}_0$.

Regression Models

- **Task:** Predict a continuous output $y \in \mathbb{R}$.

Regression Models

- **Task:** Predict a **continuous output** $y \in \mathbb{R}$.
- **One simpler model (Assume f is linear in w):** Multiple Linear Regression (MLR)

$$y_i = w_0 + w_1x_{i1} + w_2x_{i2} + \cdots + w_px_{ip} + \varepsilon_i$$

Regression Models

- **Task:** Predict a **continuous output** $y \in \mathbb{R}$.
- **One simpler model (Assume f is linear in $\mathbf{w}$):** Multiple Linear Regression (MLR)

$$y_i = w_0 + w_1x_{i1} + w_2x_{i2} + \cdots + w_px_{ip} + \varepsilon_i$$

- **Loss function (Mean Squared Error):**

$$\mathcal{L}(\mathbf{w}) = \sum_{i=1}^n (y_i - \mathbf{w}^\top \mathbf{x}_i)^2$$

Regression Models

- **Task:** Predict a **continuous output** $y \in \mathbb{R}$.
- **One simpler model (Assume f is linear in $\mathbf{w}$):** Multiple Linear Regression (MLR)

$$y_i = w_0 + w_1 x_{i1} + w_2 x_{i2} + \cdots + w_p x_{ip} + \varepsilon_i$$

- **Loss function (Mean Squared Error):**

$$\mathcal{L}(\mathbf{w}) = \sum_{i=1}^n (y_i - \mathbf{w}^\top \mathbf{x}_i)^2$$

- **Ordinary Least Squares (OLS) solution:**

$$\hat{\mathbf{w}} = (\mathbf{X}^\top \mathbf{X})^{-1} \mathbf{X}^\top \mathbf{y}$$

where

$$\mathbf{X} = \begin{bmatrix} 1 & x_{11} & x_{12} & \cdots & x_{1p} \\ 1 & x_{21} & x_{22} & \cdots & x_{2p} \\ \vdots & \vdots & \vdots & \ddots & \vdots \\ 1 & x_{n1} & x_{n2} & \cdots & x_{np} \end{bmatrix}, \quad \mathbf{y} = \begin{bmatrix} y_1 \\ y_2 \\ \vdots \\ y_n \end{bmatrix}$$

Regression Models

- **Task:** Predict a **continuous output** $y \in \mathbb{R}$.
- **One simpler model (Assume f is linear in $\mathbf{w}$):** Multiple Linear Regression (MLR)

$$y_i = w_0 + w_1x_{i1} + w_2x_{i2} + \cdots + w_px_{ip} + \varepsilon_i$$

- **Loss function (Mean Squared Error):**

$$\mathcal{L}(\mathbf{w}) = \sum_{i=1}^n (y_i - \mathbf{w}^\top \mathbf{x}_i)^2$$

- **Ordinary Least Squares (OLS) solution:**

$$\hat{\mathbf{w}} = (\mathbf{X}^\top \mathbf{X})^{-1} \mathbf{X}^\top \mathbf{y}$$

where

$$\mathbf{X} = \begin{bmatrix} 1 & x_{11} & x_{12} & \cdots & x_{1p} \\ 1 & x_{21} & x_{22} & \cdots & x_{2p} \\ \vdots & \vdots & \vdots & \ddots & \vdots \\ 1 & x_{n1} & x_{n2} & \cdots & x_{np} \end{bmatrix}, \quad \mathbf{y} = \begin{bmatrix} y_1 \\ y_2 \\ \vdots \\ y_n \end{bmatrix}$$

- **Important Note:** For all regression types of problems, we might not get the closed-form expressions of $\hat{\mathbf{w}}$.

Classification Models

- **Task:** Predict a **categorical output** $y \in \{1, 2, \dots, K\}$.

Classification Models

- **Task:** Predict a **categorical output** $y \in \{1, 2, \dots, K\}$.
- **Model:** Logistic regression (binary or multinomial)

$$P(Y = k \mid \mathbf{x}, \mathbf{w}) = \frac{\exp(\mathbf{w}_k^\top \mathbf{x})}{\sum_{j=1}^K \exp(\mathbf{w}_j^\top \mathbf{x})}$$

Classification Models

- **Task:** Predict a **categorical output** $y \in \{1, 2, \dots, K\}$.
- **Model:** Logistic regression (binary or multinomial)

$$P(Y = k \mid \mathbf{x}, \mathbf{w}) = \frac{\exp(\mathbf{w}_k^\top \mathbf{x})}{\sum_{j=1}^K \exp(\mathbf{w}_j^\top \mathbf{x})}$$

- **Loss function (Cross-Entropy / Log-Loss):**

$$\mathcal{L}(\mathbf{w}) = - \sum_{i=1}^n \sum_{k=1}^K \mathbf{1}\{y_i = k\} \log P(Y = k \mid \mathbf{x}_i, \mathbf{w})$$

Classification Models

- **Task:** Predict a **categorical output** $y \in \{1, 2, \dots, K\}$.
- **Model:** Logistic regression (binary or multinomial)

$$P(Y = k \mid \mathbf{x}, \mathbf{w}) = \frac{\exp(\mathbf{w}_k^\top \mathbf{x})}{\sum_{j=1}^K \exp(\mathbf{w}_j^\top \mathbf{x})}$$

- **Loss function (Cross-Entropy / Log-Loss):**

$$\mathcal{L}(\mathbf{w}) = - \sum_{i=1}^n \sum_{k=1}^K \mathbf{1}\{y_i = k\} \log P(Y = k \mid \mathbf{x}_i, \mathbf{w})$$

- **Estimation:** No closed-form solution; parameters $\mathbf{w}$ are found using iterative optimization methods (e.g., gradient descent, Adam, Newton–Raphson etc.)

Classification Models

- **Task:** Predict a **categorical output** $y \in \{1, 2, \dots, K\}$.
- **Model:** Logistic regression (binary or multinomial)

$$P(Y = k \mid \mathbf{x}, \mathbf{w}) = \frac{\exp(\mathbf{w}_k^\top \mathbf{x})}{\sum_{j=1}^K \exp(\mathbf{w}_j^\top \mathbf{x})}$$

- **Loss function (Cross-Entropy / Log-Loss):**

$$\mathcal{L}(\mathbf{w}) = - \sum_{i=1}^n \sum_{k=1}^K \mathbf{1}\{y_i = k\} \log P(Y = k \mid \mathbf{x}_i, \mathbf{w})$$

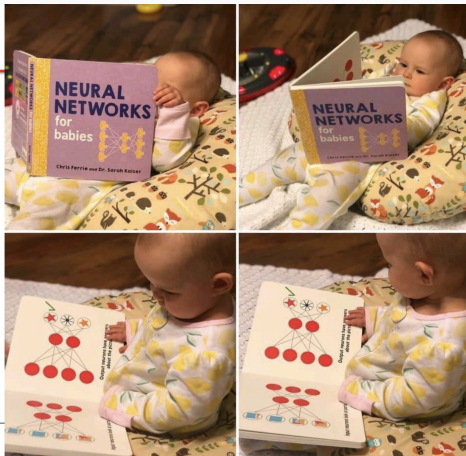
- **Estimation:** No closed-form solution; parameters $\mathbf{w}$ are found using iterative optimization methods (e.g., gradient descent, Adam, Newton–Raphson etc.)
- **Prediction:** Assign the class with the highest predicted probability

$$\hat{y}_i = \arg \max_{k \in \{1, \dots, K\}} P(Y = k \mid \mathbf{x}_i, \mathbf{w})$$

Neural Networks

Everybody is learning

Never too late



1

¹CS 229: Prof. Xiangliang Zhang (KAUST)

Agenda

2. Neural Networks

2.1 Intro to Neural Networks

2.2 Components of Neural Networks

- Layers

- Neurons, Weights, and Biases

- Activation Functions

2.3 Forward Propagation: Making Predictions

2.4 Training of NNs (Learning the Weights): Backpropagation

- Recall

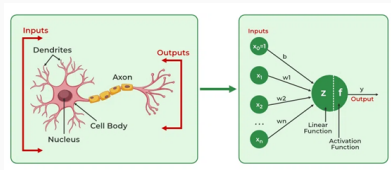
- Example: Backpropagation for two layers NNs

2.5 Building Your Very First Neural Network From Scratch

2.6 Fitting the Neural Network using Tensorflow

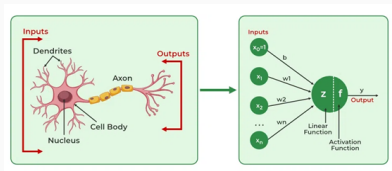
What is a Neural Network?

- Neural networks mimic how the human brain functions.



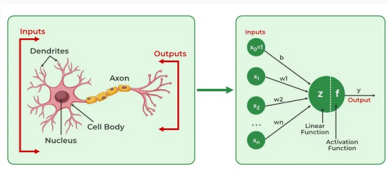
What is a Neural Network?

- Neural networks mimic how the human brain functions.
- Brain activity occurs when a **stimulus** enters the system; information is processed through the network via neurons that extract relevant information, and this information is passed to another area.



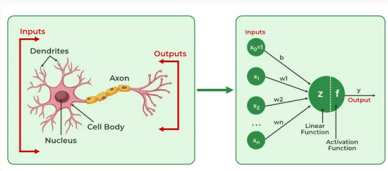
What is a Neural Network?

- Neural networks mimic how the human brain functions.
- Brain activity occurs when a **stimulus** enters the system; information is processed through the network via neurons that extract relevant information, and this information is passed to another area.
- A **Neural Network** is a computational model inspired by the way biological neural networks in the human brain process information. It consists of **layers** of interconnected **nodes**, or **neurons**, which work together to solve complex problems.



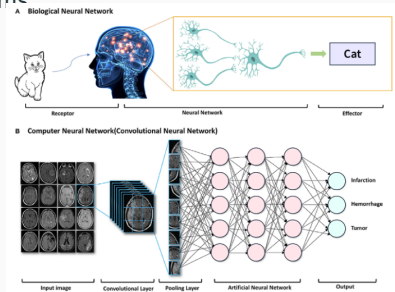
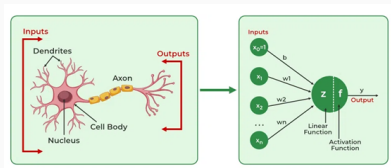
What is a Neural Network?

- Neural networks mimic how the human brain functions.
- Brain activity occurs when a **stimulus** enters the system; information is processed through the network via neurons that extract relevant information, and this information is passed to another area.
- A **Neural Network** is a computational model inspired by the way biological neural networks in the human brain process information. It consists of **layers** of interconnected **nodes**, or **neurons**, which work together to solve complex problems.



What is a Neural Network?

- Neural networks mimic how the human brain functions.
- Brain activity occurs when a **stimulus** enters the system; information is processed through the network via neurons that extract relevant information, and this information is passed to another area.
- A **Neural Network** is a computational model inspired by the way biological neural networks in the human brain process information. It consists of **layers** of interconnected **nodes**, or **neurons**, which work together to solve complex problems



Historical Development of Deep Learning

- Early Origins (1940s–1960s): McCulloch–Pitts (1943): first neuron model, Rosenblatt's Perceptron (1958): first trainable neural network.

Historical Development of Deep Learning

- **Early Origins (1940s–1960s):** McCulloch–Pitts (1943): first neuron model, Rosenblatt's Perceptron (1958): first trainable neural network.
- **First AI Winter (1970s):** Minsky & Papert (1969) show perceptron limits (XOR), Loss of funding & shift to symbolic AI.

Historical Development of Deep Learning

- **Early Origins (1940s–1960s):** McCulloch–Pitts (1943): first neuron model, Rosenblatt's Perceptron (1958): first trainable neural network.
- **First AI Winter (1970s):** Minsky & Papert (1969) show perceptron limits (XOR), Loss of funding & shift to symbolic AI.
- **Backpropagation Revolution (1980s):** Werbos (1974), Rumelhart–Hinton–Williams (1986) → training multi-layer NNs.

Historical Development of Deep Learning

- **Early Origins (1940s–1960s):** McCulloch–Pitts (1943): first neuron model, Rosenblatt's Perceptron (1958): first trainable neural network.
- **First AI Winter (1970s):** Minsky & Papert (1969) show perceptron limits (XOR), Loss of funding & shift to symbolic AI.
- **Backpropagation Revolution (1980s):** Werbos (1974), Rumelhart–Hinton–Williams (1986) → training multi-layer NNs.
- **Second AI Winter (1987–1995):** Hardware & data insufficient, neural nets sidelined; symbolic/statistical AI rise.

Historical Development of Deep Learning

- **Early Origins (1940s–1960s):** McCulloch–Pitts (1943): first neuron model, Rosenblatt's Perceptron (1958): first trainable neural network.
- **First AI Winter (1970s):** Minsky & Papert (1969) show perceptron limits (XOR), Loss of funding & shift to symbolic AI.
- **Backpropagation Revolution (1980s):** Werbos (1974), Rumelhart–Hinton–Williams (1986) → training multi-layer NNs.
- **Second AI Winter (1987–1995):** Hardware & data insufficient, neural nets sidelined; symbolic/statistical AI rise.
- **Statistical Learning Era (1990s–2000s):** SVMs (Vapnik), Probabilistic Graphical Models (Pearl), strong math foundations (optimization, RKHS).

Historical Development of Deep Learning

- **Early Origins (1940s–1960s):** McCulloch–Pitts (1943): first neuron model, Rosenblatt's Perceptron (1958): first trainable neural network.
- **First AI Winter (1970s):** Minsky & Papert (1969) show perceptron limits (XOR), Loss of funding & shift to symbolic AI.
- **Backpropagation Revolution (1980s):** Werbos (1974), Rumelhart–Hinton–Williams (1986) → training multi-layer NNs.
- **Second AI Winter (1987–1995):** Hardware & data insufficient, neural nets sidelined; symbolic/statistical AI rise.
- **Statistical Learning Era (1990s–2000s):** SVMs (Vapnik), Probabilistic Graphical Models (Pearl), strong math foundations (optimization, RKHS).
- **Deep Learning Boom (2006–2015):** DBNs (Hinton) in 2006, ImageNet/AlexNet (2012), Word2Vec, GANs.

Historical Development of Deep Learning

- **Early Origins (1940s–1960s):** McCulloch–Pitts (1943): first neuron model, Rosenblatt's Perceptron (1958): first trainable neural network.
- **First AI Winter (1970s):** Minsky & Papert (1969) show perceptron limits (XOR), Loss of funding & shift to symbolic AI.
- **Backpropagation Revolution (1980s):** Werbos (1974), Rumelhart–Hinton–Williams (1986) → training multi-layer NNs.
- **Second AI Winter (1987–1995):** Hardware & data insufficient, neural nets sidelined; symbolic/statistical AI rise.
- **Statistical Learning Era (1990s–2000s):** SVMs (Vapnik), Probabilistic Graphical Models (Pearl), strong math foundations (optimization, RKHS).
- **Deep Learning Boom (2006–2015):** DBNs (Hinton) in 2006, ImageNet/AlexNet (2012), Word2Vec, GANs.
- **Modern Era (2017–Present):** Transformers (2017), LLMs (GPT, BERT), AlphaGo, Diffusion Models, Multimodal AI.

Components of Neural Networks

Structural Components

- Layers
- Neurons
- Weights
- Bias
- Activation Functions

Components of Neural Networks

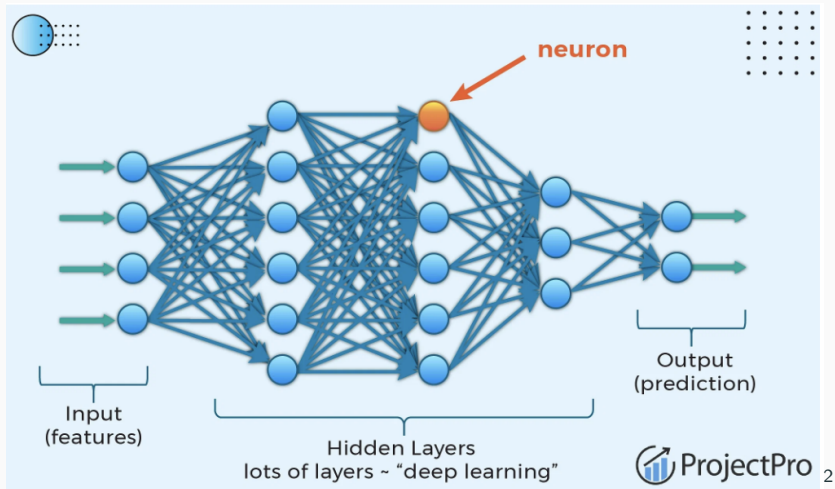
Structural Components

- Layers
- Neurons
- Weights
- Bias
- Activation Functions

Training-Related Components

- Loss Function
- Learning Rate
- Training Data
- Validation and Test Data
- Forward Propagation and Backpropagation

A Basic Architectures of Neural Networks



Agenda

2. Neural Networks

2.1 Intro to Neural Networks

2.2 Components of Neural Networks

- Layers

- Neurons, Weights, and Biases

- Activation Functions

2.3 Forward Propagation: Making Predictions

2.4 Training of NNs (Learning the Weights): Backpropagation

- Recall

- Example: Backpropagation for two layers NNs

2.5 Building Your Very First Neural Network From Scratch

2.6 Fitting the Neural Network using Tensorflow

Layers

Neural computing requires a number of neurons, to be connected together into a neural network. Neurons are arranged in layers.

- **Input Layer**

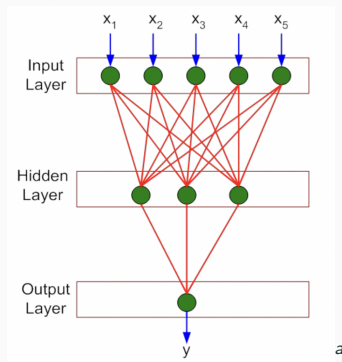
- First layer of neurons.
- Receives raw data directly from the input features.

- **Hidden Layers**

- Layers between the input and output layers.
- Perform computations.
- Can have one or many hidden layers.

- **Output Layer**

- Final layer.
- Produces the network's output.
- Structure depends on the specific task (e.g., classification, regression).



^aCS 229: Prof. Xiangliang Zhang (KAUST)

Neurons, Weights, and Biases

- **Neurons:**

- Basic units of a neural network.
- Receive input, process it, and pass it to other neurons.
- Apply a mathematical function to generate an output.

Neurons, Weights, and Biases

- **Neurons:**

- Basic units of a neural network.
- Receive input, process it, and pass it to other neurons.
- Apply a mathematical function to generate an output.

- **Weights:**

- Parameters that transform input data within the network.
- Each connection between neurons has an associated weight.

Neurons, Weights, and Biases

- **Neurons:**

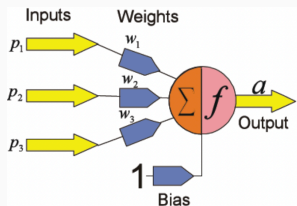
- Basic units of a neural network.
- Receive input, process it, and pass it to other neurons.
- Apply a mathematical function to generate an output.

- **Weights:**

- Parameters that transform input data within the network.
- Each connection between neurons has an associated weight.

- **Biases:**

- Additional parameters in a neuron that allow to fit the data better.
- Provide ability to each neuron have output not 0 when inputs are 0.



$$a = f(p_1w_1 + p_2w_2 + p_3w_3 + b) = f(\sum p_iw_i + b)$$

Activation Functions (1)

- Activation functions introduce **nonlinearity** into the output of a neuron, allowing neural networks to model **complex and non-linear relationships**.

Activation Functions (1)

- Activation functions introduce **nonlinearity** into the output of a neuron, allowing neural networks to model **complex and non-linear relationships**.
- While many activation functions exist, they generally satisfy two essential properties:

Activation Functions (1)

- Activation functions introduce **nonlinearity** into the output of a neuron, allowing neural networks to model **complex and non-linear relationships**.
- While many activation functions exist, they generally satisfy two essential properties:
 - **Nonlinearity**: Without nonlinear activations, the entire network reduces to a **pure linear model**, regardless of the number of layers.

Activation Functions (1)

- Activation functions introduce **nonlinearity** into the output of a neuron, allowing neural networks to model **complex and non-linear relationships**.
- While many activation functions exist, they generally satisfy two essential properties:
 - **Nonlinearity**: Without nonlinear activations, the entire network reduces to a **pure linear model**, regardless of the number of layers.
 - **Differentiability**: Required for **gradient-based optimization** methods such as backpropagation.

Activation Functions (2)

Classical Forms

Function	Expression	Pros	Cons	Uses
Sigmoid	$\frac{1}{1+e^{-x}}$	Smooth, bounded, probabilistic output	Vanishing gradients, not zero-centered	Binary classification
Softmax	$\frac{e^{x_i}}{\sum_j e^{x_j}}$	Probabilistic output vector	Sensitive to large logits	Multi-class output
Tanh	$\frac{e^x - e^{-x}}{e^x + e^{-x}}$	Zero-centered, stronger gradients	Saturates, vanishing gradients	RNNs, classical MLPs
ReLU	$\max(0, x)$	Fast, simple, no saturation for $x > 0$	Dying ReLU, unbounded output	CNNs, deep MLPs

⁴<https://www.analytixlabs.co.in/blog/activation-function-in-neural-network/>

Activation Functions (2)

Classical Forms

Function	Expression	Pros	Cons	Uses
Sigmoid	$\frac{1}{1+e^{-x}}$	Smooth, bounded, probabilistic output	Vanishing gradients, not zero-centered	Binary classification
Softmax	$\frac{e^{x_i}}{\sum_j e^{x_j}}$	Probabilistic output vector	Sensitive to large logits	Multi-class output
Tanh	$\frac{e^x - e^{-x}}{e^x + e^{-x}}$	Zero-centered, stronger gradients	Saturates, vanishing gradients	RNNs, classical MLPs
ReLU	$\max(0, x)$	Fast, simple, no saturation for $x > 0$	Dying ReLU, unbounded output	CNNs, deep MLPs

Modern Variants

Function	Expression	Pros	Cons	Uses
Leaky ReLU	$\begin{cases} x, & x > 0 \\ \alpha x, & x \leq 0 \end{cases}$	Fixes dying ReLU, small negative slope	Slope α arbitrary	CNNs, MLPs
ELU	$\begin{cases} x, & x > 0 \\ \alpha(e^x - 1), & x \leq 0 \end{cases}$	Negative outputs stabilize training	Expensive, may saturate	Deep CNNs
Swish	$x \sigma(x)$	Smooth, strong empirical performance	Slightly slower	Transformers, modern vision

⁴<https://www.analytixlabs.co.in/blog/activation-function-in-neural-network/>

Agenda

2. Neural Networks

2.1 Intro to Neural Networks

2.2 Components of Neural Networks

Layers

Neurons, Weights, and Biases

Activation Functions

2.3 Forward Propagation: Making Predictions

2.4 Training of NNs (Learning the Weights): Backpropagation

Recall

Example: Backpropagation for two layers NNs

2.5 Building Your Very First Neural Network From Scratch

2.6 Fitting the Neural Network using Tensorflow

Notation

- $\mathbf{x} \in \mathbb{R}^d$: d -dimensional input vector (features)
- $\mathbf{W}^{(l)}$: Weight matrix for layer l
- $\mathbf{b}^{(l)}$: Bias vector for layer l
- $\mathbf{z}^{(l)}$: Pre-activation (linear part)
- $\mathbf{a}^{(l)}$: Activation (output) of layer l
- $\sigma(\cdot)$: Activation function (ReLU, sigmoid, etc.)
- Total number of hidden layers will be denoted by H , and dimensions of output would be K .
- $\mathbf{t}$ is the target variables of interest.

We define $\mathbf{a}^{(0)} = \mathbf{x}$ as the input layer.

Forward Propagation: General Overview

- Forward propagation computes the **output of the neural network** by passing inputs through each layer.

Forward Propagation: General Overview

- Forward propagation computes the **output of the neural network** by passing inputs through each layer.
- Each layer performs:

$$\mathbf{z}^{(l)} = \mathbf{W}^{(l)} \mathbf{a}^{(l-1)} + \mathbf{b}^{(l)}$$

where $\mathbf{W}^{(l)}$ and $\mathbf{b}^{(l)}$ are the learnable parameters.

Forward Propagation: General Overview

- Forward propagation computes the **output of the neural network** by passing inputs through each layer.
- Each layer performs:

$$\mathbf{z}^{(l)} = \mathbf{W}^{(l)} \mathbf{a}^{(l-1)} + \mathbf{b}^{(l)}$$

where $\mathbf{W}^{(l)}$ and $\mathbf{b}^{(l)}$ are the learnable parameters.

- The activation function transforms the linear output:

$$\mathbf{a}^{(l)} = \sigma(\mathbf{z}^{(l)})$$

Forward Propagation: General Overview

- Forward propagation computes the **output of the neural network** by passing inputs through each layer.
- Each layer performs:

$$\mathbf{z}^{(l)} = \mathbf{W}^{(l)} \mathbf{a}^{(l-1)} + \mathbf{b}^{(l)}$$

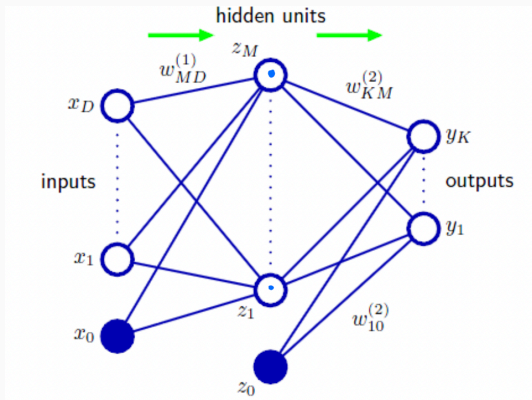
where $\mathbf{W}^{(l)}$ and $\mathbf{b}^{(l)}$ are the learnable parameters.

- The activation function transforms the linear output:

$$\mathbf{a}^{(l)} = \sigma(\mathbf{z}^{(l)})$$

- This process continues layer by layer until the final output is produced.

Example: Unfolding a Single-Hidden-Layer (Two-Layer) Neural Network



- D input variables
- M number of hidden neurons
- K outputs
- Use the two activation functions on the weights sum on hidden layers and output layers

A Two-Layer Neural Network (2)

Construct M linear combinations of the inputs $x_1, \dots, x_d$

$$z_j^{(1)} = \sum_{i=0}^D w_{ji}^{(1)} x_i, \quad x_0 = 1.$$

- $z_j^{(1)}$ are the **pre-activations** on j^{th} neurons of hidden layers, $j = 1, \dots, M$.
- $w_{ji}^{(1)}$ are the layer one weights, $i = 1, \dots, D$.
- $w_{j0}^{(1)}$ are the layer one biases.

A Two-Layer Neural Network (2)

Construct M linear combinations of the inputs $x_1, \dots, x_d$

$$z_j^{(1)} = \sum_{i=0}^D w_{ji}^{(1)} x_i, \quad x_0 = 1.$$

- $z_j^{(1)}$ are the **pre-activations** on j^{th} neurons of hidden layers, $j = 1, \dots, M$.
- $w_{ji}^{(1)}$ are the layer one weights, $i = 1, \dots, D$.
- $w_{j0}^{(1)}$ are the layer one biases.

Each linear combination $z_j^{(1)}$ is transformed by a (non-linear differentiable) activation function,

$$a_j^{(1)} = h(z_j^{(1)}),$$

which will be the **activation** now at the j^{th} neurons of the hidden layer and act as an input for the next layers (here output).

A Two-Layer Neural Network (3)

The hidden outputs $a_j^{(1)} = h(z_j^{(1)})$ are linearly combined in layer two:

$$z_k^{(2)} = \sum_{j=0}^M w_{kj}^{(2)} a_j^{(1)}, \quad z_0 = 1.$$

- $z_k^{(2)}$ are the output pre-activations at the k^{th} output neurons, $k = 1, \dots, K$.
- $w_{kj}^{(2)}$ are the layer two weights, $j = 1, \dots, M$.
- $w_{k0}^{(2)}$ are the layer one biases.

A Two-Layer Neural Network (3)

The hidden outputs $a_j^{(1)} = h(z_j^{(1)})$ are linearly combined in layer two:

$$z_k^{(2)} = \sum_{j=0}^M w_{kj}^{(2)} a_j^{(1)}, \quad z_0 = 1.$$

- $z_k^{(2)}$ are the output pre-activations at the k^{th} output neurons, $k = 1, \dots, K$.
- $w_{kj}^{(2)}$ are the layer two weights, $j = 1, \dots, M$.
- $w_{k0}^{(2)}$ are the layer one biases.

The output pre-activations $z_k^{(2)}$ are transformed by the output (non-linear differentiable) activation function

$$y_k = \sigma(z_k^{(2)}).$$

- y_k are the final outputs.
- $\sigma(\cdot)$ is like $h(\cdot)$, but often sigmoid function for classification and linear function for Regressions.

A Two-Layer Neural Network (4)

After substituting $y_k = \sigma(z_k^{(2)})$;

$$y_k(\mathbf{x}, \mathbf{w}) = \sigma \left(\sum_{j=0}^M w_{kj}^{(2)} h \left(\sum_{i=0}^D w_{ji}^{(1)} x_i \right) \right).$$

A Two-Layer Neural Network (4)

After substituting $y_k = \sigma(z_k^{(2)})$;

$$y_k(\mathbf{x}, \mathbf{w}) = \sigma \left(\sum_{j=0}^M w_{kj}^{(2)} h \left(\sum_{i=0}^D w_{ji}^{(1)} x_i \right) \right).$$

Evaluation of this is called **forward propagation**.

- Here, $h(\cdot)$ and $\sigma(\cdot)$ are activation functions.
- There could be several possibilities of activation function; for example:
 - If $\sigma(\cdot)$ is the **sigmoid function**, then a **binary classification model** is obtained, despite any choice of activation $h(\cdot)$
 - If $\sigma(\cdot)$ is the **identity function**, then a **regression model** is obtained.
 - If $\sigma(\cdot)$ **both are identity functions**, then we get a **Linear regression**; for example, MLR

Exercise: Three-Layer Neural Network (Forward Propagation)

Similar to two-layer neural network show that the network output for a three layer-neural network would be

$$\hat{y}_k(\mathbf{x}, \mathbf{w}) = \sigma \left(\sum_{m=0}^{M_2} w_{km}^{(3)} h^{(2)} \left(\sum_{j=0}^{M_1} w_{mj}^{(2)} h^{(1)} \left(\sum_{i=0}^D w_{ji}^{(1)} x_i \right) \right) \right).$$

Here:

- $w_{ji}^{(1)}$ are weights from input components x_i to first hidden layer neuron j (with $i = 0$ representing bias and $x_0 = 1$).
- $h^{(1)}$ and $h^{(2)}$ are the activation functions for hidden layer 1 and hidden layer 2 respectively.
- $w_{mj}^{(2)}$ are weights from first hidden layer (j) to second hidden layer (m); $w_{km}^{(3)}$ are weights from second hidden layer (m) to output unit k (again $m = 0$ may be bias).
- D = input dimension, M_1 = number of units in first hidden layer, M_2 = number of units in second hidden layer.

Special cases (Think about these as well):

- If $h^{(1)}$, $h^{(2)}$, σ are *identity*, the whole map reduces to a single affine function of $\mathbf{x}$ (linear regression).
- If σ is softmax and the hidden activations are non-linear, this is a multi-class classification model.

A Nice Visual of NNs Architectures

Let's have some fun: <https://playground.tensorflow.org/>

Agenda

2. Neural Networks

2.1 Intro to Neural Networks

2.2 Components of Neural Networks

- Layers

- Neurons, Weights, and Biases

- Activation Functions

2.3 Forward Propagation: Making Predictions

2.4 Training of NNs (Learning the Weights): Backpropagation

- Recall

- Example: Backpropagation for two layers NNs

2.5 Building Your Very First Neural Network From Scratch

2.6 Fitting the Neural Network using Tensorflow

Recall: Error (Loss) Function

Training a neural network involves estimating the weights and biases by optimizing a loss function.

- Regression: Sum of squared

$$\mathcal{L}(\mathbf{w}) = \frac{1}{2} \sum_{n=1}^N \{y(\mathbf{x}_n, \mathbf{w}) - t_n\}^2,$$

where $t_n, n = 1, \dots, N$ is the observed response. N is the total number of training samples (examples).

Recall: Error (Loss) Function

Training a neural network involves estimating the weights and biases by optimizing a loss function.

- **Regression: Sum of squared**

$$\mathcal{L}(\mathbf{w}) = \frac{1}{2} \sum_{n=1}^N \{y(\mathbf{x}_n, \mathbf{w}) - t_n\}^2,$$

where $t_n, n = 1, \dots, N$ is the observed response. N is the total number of training samples (examples).

- **Multi-class classification:** Neural networks with a softmax output layer use the **Cross-entropy loss**:

$$\mathcal{L}(\mathbf{w}) = - \sum_{n=1}^N \sum_{k=1}^K t_{nk} \log y_k(\mathbf{x}_n, \mathbf{w}),$$

where:

- K = number of classes,
- $t_{nk} = 1$ if sample n belongs to class k , else 0,
- $y_k(\mathbf{x}_n, \mathbf{w})$ = softmax probability for class k .

Training a Neural Network: Forward and Backward Pass

- **Goal:** Minimize the error/loss function $\mathcal{L}(\mathbf{w})$
- **Recall:** We will be using gradient based algorithm for minimizing the loss function which require calculation of gradient of loss function.
- **Each training step has two key phases:**
 - **Forward Pass:** Compute output predictions from input through the layers. (We covered this part previously)
- Then update weights using **gradient descent** (or a variant).

Training a Neural Network: Forward and Backward Pass

- **Goal:** Minimize the error/loss function $\mathcal{L}(\mathbf{w})$
- **Recall:** We will be using gradient based algorithm for minimizing the loss function which require calculation of gradient of loss function.
- **Each training step has two key phases:**
 - **Forward Pass:** Compute output predictions from input through the layers. (We covered this part previously)
 - **Backward Pass (Backpropagation):** Compute gradients of loss w.r.t. weights using the chain rule.
- Then update weights using **gradient descent** (or a variant).

Backpropagation: overview

- **Goal:** compute $\nabla_{\mathbf{w}} \mathcal{L}(\mathbf{y}(\mathbf{x}, \mathbf{w}), \mathbf{t})$ efficiently for all **weights** in the network.
- Use **chain rule** to propagate “**errors**” from the **output layer** to backward through layers.
- For example, for a **two-layer NNs** (input $\rightarrow$ 1-hidden layer $\rightarrow$ output) we compute:

$$\frac{\partial \mathcal{L}}{\partial w_{kj}^{(2)}}, \quad \frac{\partial \mathcal{L}}{\partial w_{ji}^{(1)}}$$

where indices follow the earlier slides in case of forward propagation:

$$z_j^{(1)} = \sum_{i=0}^D w_{ji}^{(1)} x_i, \quad x_0 = 1, \quad a_j^{(1)} = h(z_j^{(1)}), \quad j = 1, \dots, M$$

$$z_k^{(2)} = \sum_{j=0}^M w_{kj}^{(2)} a_j^{(1)}, \quad a_0^{(1)} = 1, \quad y_k = \sigma(z_k^{(2)}), \quad k = 1, \dots, K.$$

Common activation derivatives

- **Sigmoid** $\sigma(u) = \frac{1}{1+e^{-u}}$: $\sigma'(u) = \sigma(u)(1 - \sigma(u))$.
- **Tanh**: $h(u) = \tanh(u)$: $h'(u) = 1 - \tanh^2(u)$.
- **ReLU**: $h(u) = \max(0, u)$: $h'(u) = \begin{cases} 1 & u > 0 \\ 0 & u < 0 \end{cases}$ (undefined at 0, set 0 or 1 in practice).
- **Leaky ReLU**: $h'(u) = 1$ if $u > 0$, else α .

Note: compute derivatives using **pre-activation values** z (numerically stable).

Agenda

2. Neural Networks

2.1 Intro to Neural Networks

2.2 Components of Neural Networks

- Layers

- Neurons, Weights, and Biases

- Activation Functions

2.3 Forward Propagation: Making Predictions

2.4 Training of NNs (Learning the Weights): Backpropagation

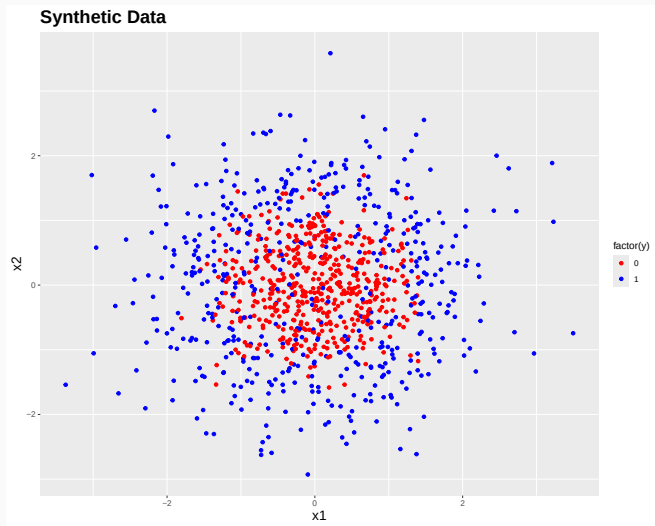
- Recall

- Example: Backpropagation for two layers NNs

2.5 Building Your Very First Neural Network From Scratch

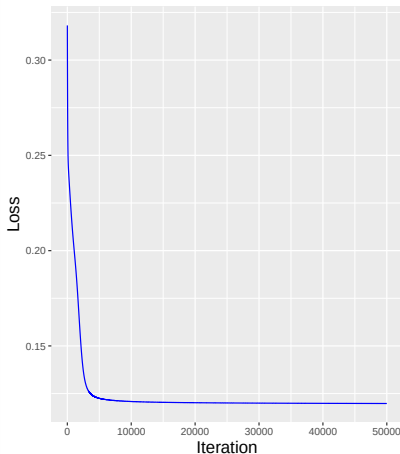
2.6 Fitting the Neural Network using Tensorflow

Lab: Classification Using two-Layer Neural Networks (1)

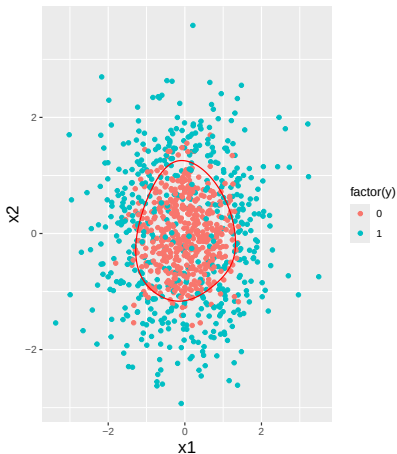


Lab: Classification Using two-Layer Neural Networks (2)

Loss Function Over Iterations



Decision Boundary of the Neural Network



⁵See the python code in the GitHub Link:

https://github.com/yadavrishikesh/Masterclass-V2-2026/blob/main/module2/code/NNs_for_classification_from_scratch.ipynb

Agenda

2. Neural Networks

2.1 Intro to Neural Networks

2.2 Components of Neural Networks

- Layers

- Neurons, Weights, and Biases

- Activation Functions

2.3 Forward Propagation: Making Predictions

2.4 Training of NNs (Learning the Weights): Backpropagation

- Recall

- Example: Backpropagation for two layers NNs

2.5 Building Your Very First Neural Network From Scratch

2.6 Fitting the Neural Network using Tensorflow

What is TensorFlow

TensorFlow is an open-source machine learning library developed by Google. It provides a comprehensive, flexible ecosystem of tools, libraries, and community resources that lets researchers push the state-of-the-art in ML, and developers easily build and deploy ML-powered applications.

What is TensorFlow

TensorFlow is an open-source machine learning library developed by Google. It provides a comprehensive, flexible ecosystem of tools, libraries, and community resources that lets researchers push the state-of-the-art in ML, and developers easily build and deploy ML-powered applications.

- Originally developed by the Google Brain Team
- Supports deep learning, numerical computation, and automatic differentiation
- Works across multiple platforms: CPUs, GPUs, TPUs

Typical steps in building a model with TensorFlow:

1. **Prepare** the data
2. **Build** the model (using `keras.Sequential`)
3. **Compile** the model (using `model.compile`) and mention about the loss, optimizer, and metrics
4. **Train** the model (`model.fit`)
5. **Evaluate** the model (`model.evaluate`)
6. **Predict** with new data (`model.predict`)

Why Use TensorFlow?

- **Scalability:** Efficient execution across devices and distributed systems
- **Production Ready:** Tools for deploying models to web, mobile, and edge devices
- **Ecosystem:** Strong support for [visualization](#), [data pipelines](#), and [deployment](#)
- **Community:** Backed by Google and widely adopted in academia and industry

[TensorFlow](#) is designed to support research prototyping and production deployment equally well.

Key Steps in Fitting a Neural Network (TensorFlow)

- **Step 1: Load and Prepare Data**

- Use `tensorflow.keras.datasets` or custom datasets.
- Normalize/scale input features.

- **Step 2: Define the Model Architecture**

- Use `keras.Sequential` or subclassing API.
- Stack layers using `Dense`, `Dropout`, etc.

- **Step 3: Compile the Model**

- Specify `loss function`, `optimizer`, and `metrics`.

Model Definition Example (TensorFlow)

- A simple feedforward network for binary classification:

```
1 model = keras.Sequential([
2     keras.layers.Input(shape=(num_features,)),
3     keras.layers.Dense(8, activation='relu'),
4     keras.layers.Dense(10, activation='relu'),
5     ...
6     keras.layers.Dense(1, activation='sigmoid')
7 ])
```

Model Definition Example (TensorFlow)

- A simple feedforward network for binary classification:

```
1 model = keras.Sequential([  
2     keras.layers.Input(shape=(num_features,)),  
3     keras.layers.Dense(8, activation='relu'),  
4     keras.layers.Dense(10, activation='relu'),  
5     ...  
6     keras.layers.Dense(1, activation='sigmoid')  
7 ])
```

- `input_shape`: Number of input features.
- `activation='relu'`: Adds non-linearity.
- `activation='sigmoid'`: For binary classification.

Compiling the Model

```
1 model.compile(  
2     optimizer='adam',  
3     loss='binary_crossentropy',  
4     metrics=['accuracy']  
5 )
```

Compiling the Model

```
1 model.compile(  
2     optimizer='adam',  
3     loss='binary_crossentropy',  
4     metrics=['accuracy']  
5 )
```

- optimizer: Controls weight updates (e.g., adam, sgd).
- loss: Chosen based on task:
 - binary_crossentropy for binary classification
 - categorical_crossentropy for multi-class
 - mse for regression
- metrics: To monitor model performance during training.

Training and Evaluation

```
1 history = model.fit(  
2     X_train, y_train,  
3     validation_data=(X_val, y_val),  
4     epochs=50,  
5     batch_size=32  
6 )
```

Training and Evaluation

```
1 history = model.fit(  
2     X_train, y_train,  
3     validation_data=(X_val, y_val),  
4     epochs=50,  
5     batch_size=32  
6 )
```

- `validation_data`: Used to monitor generalization.
- `epochs`: Number of training passes over data.
- `batch_size`: Mini-batch size.

```
1 test_loss, test_acc = model.evaluate(X_test, y_test)
```

Lab: Fitting the Neural Network using Tensorflow

See the GitHub Link:

https://github.com/yadavrishikesh/Masterclass-V2-2026/blob/main/module2/code/NNs_for_classification_using_tensorflow.ipynb